

Reschedule: Modal vs. Creation Wizard Page

UX discussion prep — Development team review, August 7, 2026

Executive summary

The reschedule flow proposed by design uses a modal that opens on top of the job page, keeping the user's full context (order, status, vehicles) visible behind it. The alternate proposal is to reuse the calendar step that already exists inside the job creation wizard, landing on it directly when the user clicks “Reschedule” from an existing job.

This document lays out why keeping the modal is the right call, and details the concrete problems — both UX and flow-architecture — that the wizard-reuse proposal introduces.

Why the modal is the right choice

1. Context preservation

At every step of the proposed flow, the background remains the same job page: same header, same Order #, same vehicle table. The user never loses sight of where they are and never needs to reload or re-navigate to recover context. With a separate page, every reschedule means leaving the job, entering a different route, and coming back — breaking scroll position, breadcrumb navigation, and the mental model of “I'm working on this job.”

2. Reschedule is a sub-task, not a destination

The user's goal is still to manage the job; rescheduling is a brief interruption to that goal, not a new place they want to “go.” Modals are the right pattern for focused tasks that return the user to where they were. Full pages are for when the user genuinely wants to stay there — like Manual Schedule, which earns being a page because it's a longer, more open-ended process.

3. Established precedent in the product

Cancel Job, Edit Installation Address, and Edit Site Contact are already modals within this same flow. If Reschedule suddenly navigates to a full page, it breaks interaction consistency: the user has to learn two different mental models for actions that are conceptually the same type — adjusting one aspect of an existing job.

4. Error handling needs to live in the same place

The reschedule flow includes an explicit decision point when the API call (VPM/ERP) fails: the user can retry or go to manual reschedule. This only reads as “a hiccup in what I'm doing” if it happens in the same container where the task started. On a separate page, a failure has no clean place to “go back” to.

5. Perceived reversibility

A modal implicitly communicates that it's easy to back out (the X, the Cancel button, always visible). Navigating to a full page implies a bigger commitment and requires an explicit action — browser back, breadcrumb — to change your mind midway.

6. Speed for the highest-frequency use case

Reschedule is a high-frequency action for agents managing multiple jobs a day. Every full-page navigation (load, breadcrumb, recovering position in the vehicle table) is accumulated friction. The modal reduces how many full page loads a repeat user sits through in a day.

7. It's the pattern already being implemented for every job action

Every action that lives at the top of the job page (Cancel Job, Reschedule, Edit Installation Address, Edit Site Contact) is being built as a modal, specifically to keep the user's context inside the job at all times. This isn't an isolated choice for reschedule — it's the interaction pattern already defined for this entire area of actions. Pulling only reschedule out of that pattern, and sending it to a different page, breaks the consistency the rest of the design is already building.

8. Minimal action surface — no distractions

The modal contains only the calendar and the actions reschedule needs: pick a date, pick a time, confirm. Nothing else. It is exactly what the agent needs to complete the task, and nothing that distracts from it. Landing on the creation wizard page instead exposes the agent to many other buttons and controls that have nothing to do with rescheduling (scope editing, address changes, navigation between creation steps) — unnecessary interaction surface that raises the odds an agent clicks something they shouldn't, or simply loses focus from the one goal of the task.

On the argument that the modal is “something new”

One argument raised is that choosing the modal means building something new, instead of reusing something already known. That argument doesn't hold up once the flow is looked at closely.

The reschedule modal doesn't invent a new calendar or a new time-slot picker: it uses the exact same calendar component and the same slot components that already exist and are used on the creation wizard page. The only thing that changes is the container — modal instead of page navigation — not the content, not the availability logic, not the calendar's appearance.

And that container isn't new either: the modal is the same pattern already being used and validated for Cancel Job, Edit Installation Address, and Edit Site Contact within this same product. It is, in fact, the more familiar and proven pattern in this specific context — not the less familiar one.

What would genuinely be new and untested is entering the creation wizard directly at its calendar step, without having gone through the previous steps — that is the interaction nobody has designed or tested yet.

Concrete problems with the alternate proposal

Landing directly on the calendar step of the job creation wizard isn't just a different UX preference: it introduces technical and experience risks worth putting on the table.

1. Landing on that step reopens fields reschedule was never meant to touch

The calendar step, inside the creation flow, assumes the user already went through vehicle scope and the installation-address decision (see the decision diamond and the “Different Installation Address” branch in the

creation flow). For an existing job, that data does already exist — it's how the job got created in the first place. The problem isn't missing data; it's that landing directly on that step, inside the same wizard, keeps the door open to editing scope or address along the way, even though reschedule isn't supposed to touch either one. Isolating just the calendar for reschedule means it can no longer sit inside the same wizard step that also allows those edits.

2. “Back” and “Cancel” don't mean the same thing in a reschedule

In job creation, “Back” returns to the previous wizard step (address, vehicle scope) and “Cancel” likely aborts the entire job creation. If a user lands on that same step from reschedule, what does “Back” do? There's a real risk the user ends up editing vehicle scope or the address of an existing job they never intended to touch — that behavior, inherited from the wizard, doesn't apply to the reschedule context.

3. Reschedule's error flow has nowhere to live inside a linear step

The creation wizard is designed to move forward linearly toward “Booking confirmed” — it has no concept of “this failed, choose between retry or manual”, which reschedule needs. Bolting that decision logic onto a step built for creation forces a behavior it was never designed to support.

4. Header/breadcrumb inconsistent with the actual action

The creation wizard's header and breadcrumb likely communicate “creating a new job.” Arriving there from “Reschedule” on a job that already exists can send a confusing visual signal — the user might wonder if they're accidentally creating something new.

5. Exposes controls that have nothing to do with reschedule

The wizard page brings along every other step and control from the creation flow (vehicle scope editing, address changes, navigation between steps). Reschedule only needs the calendar and confirming the new time — any additional control on screen is unnecessary surface for error: an agent moving quickly between jobs is more likely to click something they shouldn't, or lose focus from the single goal of the task.

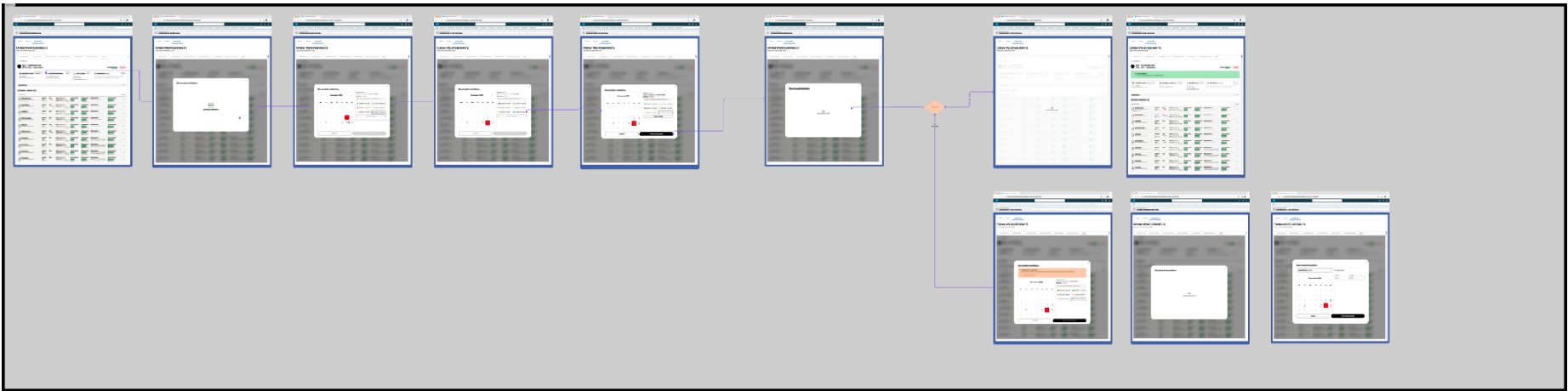
The central point

What's genuinely worth reusing is the calendar COMPONENT and its availability logic — not the entire PAGE, with its wizard, its breadcrumb, and its “you're creating a job” semantics. If the calendar step has to be decoupled from the rest of the wizard to work as a direct entry point, it's already being decoupled from the page anyway. At that point, mounting it inside a modal is no more work than turning it into a standalone route — and it avoids inheriting the wizard's back/cancel behavior, which doesn't apply to reschedule.

Recommendation

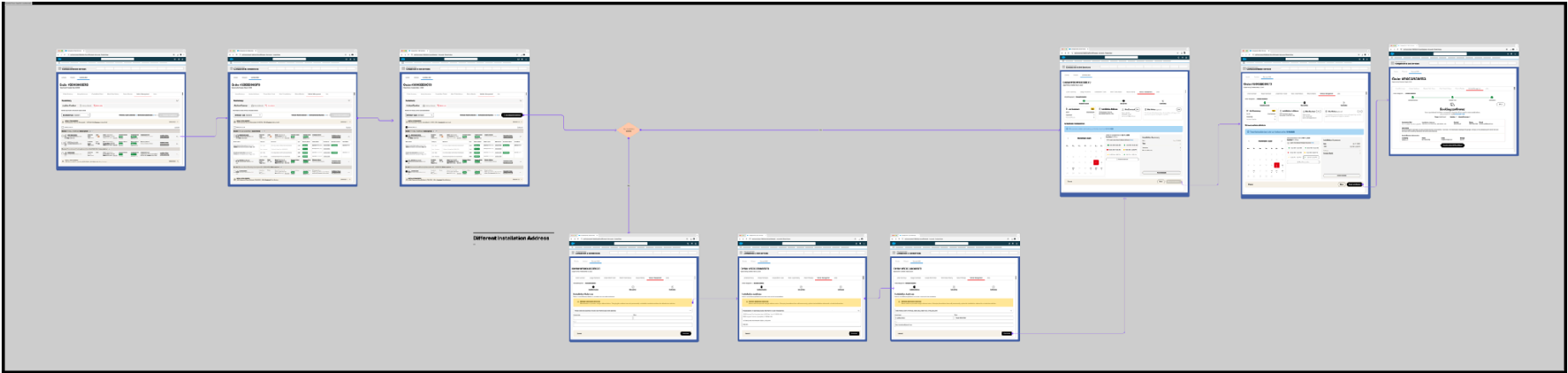
Keep the modal as the container for reschedule, reusing the calendar and availability component (UI and logic) that already exists in the creation wizard through a shared interface between both contexts. This gives development the code reuse it's after, without sacrificing interaction consistency or inheriting navigation behavior (back/cancel/breadcrumb) that doesn't apply to the task of rescheduling an existing job.

Appendix A — Proposed reschedule flow (modal)



Every step keeps the same job page visible behind the modal. Error state (red) branches to retry or manual reschedule via the decision diamond.

Appendix B — Existing job creation flow (wizard)



Linear, multi-step wizard ending in “Booking confirmed.” The calendar step (steps highlighted in the sequence) assumes prior steps — scope and installation address — have already been completed.